vllm-sr/Vela-2.0-4B

Like 13
Follow vLLM Semantic Router 410

Zero-Shot Classification | Safetensors | 19 datasets | 17 languages | vela2-decoder | decision-model | system-one | qwen3.5 | semantic-router | vela | token-classification | named-entity-recognition | pii-detection | hallucination-detection | content-safety | multilingual | long-context | custom_code | License: apache-2.0

→ Copy to bucket NEW



Docs | Blog | GitHub | Vela 2.0 collection

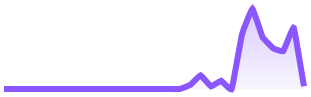
Vela 2.0 4B

Open Foundation Routing Models

Routing decisions. Safety checks. Precise text spans.

The balanced hybrid member of Vela 2.0: strong multilingual safety and general decisions, with router and open-label spans through one interface.

Downloads last month
263



Safetensors

Model size	4B params
Tensor type	F32 · BF16

Files info

Inference Providers NEW

Zero-Shot Classification

This model isn't deployed by any Inference Provider.

Ask for provider support

Model tree for vllm-sr/Vela-2.0-…

Base model — Qwen/Qwen3.5-4B-Base
Finetuned — vllm-sr/Decision-2…
Finetuned (2) — this model
Finetunes — 1 model

Datasets used to train vllm-sr/Ve…

Define options, labels and rubrics at request time. Ask multiple named questions about a request, context and answer, and receive structured decisions with the text spans that support your workflow.

1. **Balanced routing capability.** Safety macro AUC of 0.921 across 14 public sets, alongside open-label extraction and general decisions.
2. **Routing and safety together.** Use one request for routing, prompt-attack checks, PII and unsupported-claim detection.
3. **Decisions at span resolution.** Return precise text locations for trained router labels and open-label extraction, with probabilities and character offsets.

Specification	Value
Reported parameters	4.2B
Backbone	32-layer Qwen3.5 hybrid: Gated DeltaNet and gated GQA
Input limit	16,384 tokens
Output	Choice, Yes/no (Noul), Score, Span and Set; no text generation
Evaluated precision	FP32 parameters; bf16 backbone autocast on GPU; FP32 heads
GPU parameter memory	About 17 GB in FP32
Licence	Apache-2.0

Quickstart

hotpotqa/hotpot_qa
Viewer • • 203k • 98.2k • 34(

rajpurkar/squad_v2
Viewer • l • 142k • 46k • 267

nvidia/Aegis-AI-Content-Saf…
Viewer • • 33.4k • 14.6k • 11

Space using `vllm-sr/Vela-2.0-…` 1

vllm-sr/vela2-studio

Collection including `vllm-sr/Vela…`

Vela 2.0 Collection
Towards O… • 5 items • Upd… • 30

Install the dependencies, load the model and ask for routing and spans in one request:

For focused examples, see Span questions and Set questions below.

```
pip install torch "transformers>=5.17" safet
pip install flash-linear-attention   # GPU:


from transformers import AutoModel

m = AutoModel.from_pretrained("vllm-sr/Vela-
m = m.to("cuda")   # parameters load in FP32

PII_LABELS = m.vela2_engine.cal["pii_schema"]

result = m.system_one(
    state={"request": "Hi, I'm Tom Baker (to
           "source": "For adults, the maximu
           "answer": "Adults can take up to
    questions={
        "pii": {"type": "span", "instruction
                "over": "request"},
        "halu": {"type": "span", "instructio
                 "criteria": {"unsupported":
        "domain": {"type": "choice", "instru
                   "criteria": {"health": "m
                                "math": "ari
                                "other": "a
    })
```

Recorded response excerpt from this release; the full response includes usage and thresholds:

```
{
  "answers": {
    "domain": {
      "type": "choice",
```

```
      "choice": "health",
      "confidence": 0.975,
      "probabilities": {
        "health": 0.987,
        "math": 0.001,
        "other": 0.012
      }
    },
  },
  "spans": {
    "pii": [
      {
        "label": "PERSON",
        "start": 8,
        "end": 17,
        "text": "Tom Baker",
        "probability": 0.927
      },
      {
        "label": "EMAIL_ADDRESS",
        "start": 19,
        "end": 40,
        "text": "tom.baker@example.com",
        "probability": 1.0
      }
    ],
    "halu": [
      {
        "label": "unsupported",
        "start": 19,
        "end": 29,
        "text": "to 6 grams",
        "probability": 0.813
      }
    ]
  },
  "span_heads": {
    "pii": "router",
    "halu": "router"
  }
}
```

GPU with bf16 autocast is the evaluated setting. FP32 on CPU or GPU is also supported; fp16 is not. Parameter memory above excludes runtime allocations. See precision and execution.

Span questions: locate text

Use `"type": "span"` to extract text with labels and character offsets. Set `criteria` to a label-to-description mapping and `over` to the state field to inspect. This PII example reuses `m` and the trained `PII_LABELS` from above:

```
span_result = m.system_one(
    state={"request": "Hi, I'm Tom Baker (tor
    questions={
        "pii": {
            "type": "span",
            "instructions": "Which spans are
            "criteria": PII_LABELS,
            "over": "request",
        }
    },
)
print(span_result["spans"]["pii"])
```

Each returned span includes `label`, `text`, `start`, `end` and `probability`. Offsets use Unicode code points, with `start` inclusive and `end` exclusive, in the selected field. The first example also shows hallucination detection: use the `unsupported` label, target `over="answer"`, and supply grounding text in `state["source"]`.

Two span heads, one interface. The router span head handles trained PII, hallucination and toxic labels. The broad span head handles open-label

extraction. The engine selects a head for each span question; you can also set `"head": "router"` or `"head": "broad"`. See span-head selection. For entities, relations and evidence examples, see open-label extraction.

Set questions: select multiple labels

Use `"type": "set"` when zero, one or several labels can apply. Supply your labels and descriptions in `criteria`; each label is scored independently. This example reuses `m` from above:

```
set_result = m.system_one(
    state="My card was charged twice and the
    questions={
        "issues": {
            "type": "set",
            "instructions": "Which issues do
            "criteria": {
                "billing": "payments, charge
                "shipping": "delivery of an
                "login": "signing in, passwo
            },
        }
    },
)
print(set_result["sets"]["issues"]["selected
print(set_result["sets"]["issues"]["probabil
```

Read the selected labels and all per-label probabilities from `sets["issues"]`. Probabilities do not need to sum to one. Labels are selected when their probability exceeds the question's threshold; omit `threshold` to use shipped calibration, or add it to the question to set your own cutoff. The applied value is returned in `set_result["thresholds"]["issues"]`.

Questions and outputs

Question	Use it for	Output
Choice	Route a request or select one of 2–255 supplied options.	Selected key and distribution
Yes/no (Noul)	Check a condition against the state.	P(yes)
Score	Rate against 2–10 ordered levels.	Expected level and distribution
Span	Locate personal information, unsupported claims, entities, relations or evidence.	Text spans, character offsets and probabilities
Set	Select any number of supplied labels.	Selected labels and per-label probabilities

Span offsets refer to Unicode code points in the selected text. Span and Set answers also provide Yes/no views in the SystemOne response. Output semantics and thresholds explain these fields.

Results

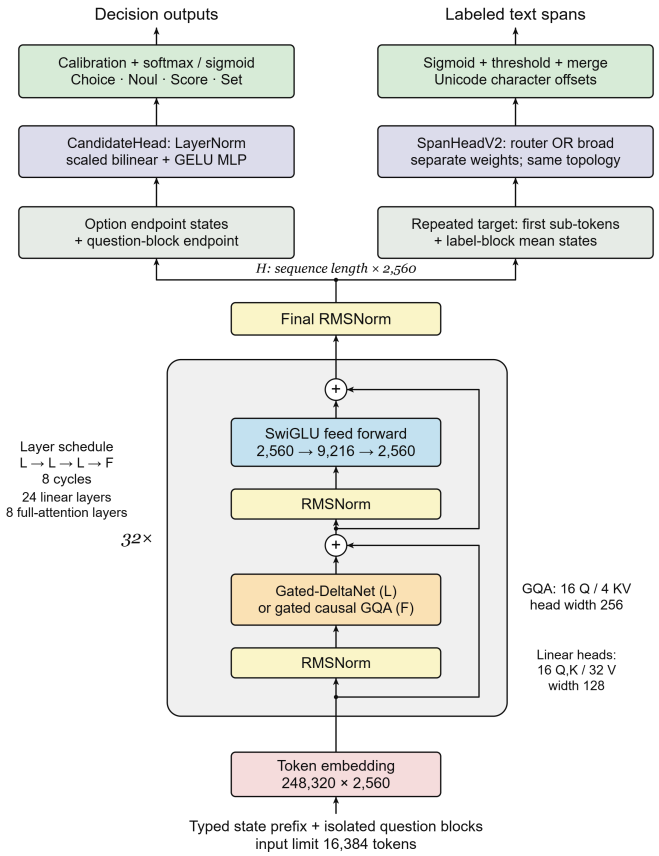
Selected results for this checkpoint. Full evaluation includes every benchmark, family comparison and scoring protocol.

Benchmark	Result	Metric and mode
Safety, 14 public sets	**0.921**	Mean AUC; trained task families

Benchmark	Result	Metric and mode
Prompt attacks, unseen families	**0.985**	AUC
PII, short texts	**0.985**	F1; shipped calibration
ACL-Verbatim evidence	**24.4**	Word-F1; broad span head, zero-shot
Jev Decision Index 0.2.1	**31.91**	38 benchmarks; `noul_calibration=True`

- **Evaluation:** checkpoint selection, temperatures and thresholds used dev data. Safety results cover trained task families; they are not zero-shot comparisons.
- **Calibration:** Jev uses optional `noul_calibration=True`; the default score is 31.63. PII uses shipped calibration. All evaluation modes.

Architecture



Vela 2.0 4B

Causal tree execution; the state prefix is reused per layer. Head computations use FP32.

This model has its own 32-layer hybrid backbone, with Gated DeltaNet and gated GQA blocks, a candidate head, and separate router/broad span-head weights. Editable SVG.

▶ Operator and readout diagrams

Choice, Yes/no, Score and Set share a candidate readout. Router and broad span heads use the same word-by-label topology with separate weights. The hybrid backbone reuses the state prefix within each rendered sequence; execution details are in USAGE.md.

Reference

Document	Contents
USAGE.md	Full examples and recorded responses, typed parts, head selection, calibration, SDK and HTTP serving
EVALUATION.md	Complete family results, research versus shipped PII modes, benchmark protocols and evaluation disclosures
TRAINING.md	Training stages, data mixtures, provenance, licences and release files
PARITY.md	Export parity against the research scorer

Credit and citation

Vela 2.0 is led by KR Labs and vLLM Semantic Router.

Read the Vela 2.0 technical overview.

```
@misc{vela2_4b_2026,
  title        = {Vela 2.0: Towards Open Fou
  author       = {{KR Labs} and {vLLM Semant
  year         = {2026},
  note         = {Blog post. Model: vllm-sr/
  howpublished = {\url{https://vllm-sr.ai/bl
}
```

System theme TOS Privacy About Careers Models Datasets Spaces Pricing Docs

Licence

Apache-2.0 for this model's weights, code and documentation. It is derived from Decision-2.0-Nox-4B (Apache-2.0), itself built on Qwen3.5-4B (Apache-2.0); their licences and notices are passed on in `LICENSE`, `NOTICE`, `ATTRIBUTIONS.md` and `LICENSES/`. Changes against Decision-2.0-Nox-4B:

MODIFICATIONS.md. Training data keep their own licences, some of them CC-BY-SA share-alike.